Ch02: 物件導向設計

繼承、多型與介面

講師: [薛念林]

日期: 2026-03-09

學習目標

1. 理解 **繼承 (Inheritance)** 的應用與建構子設計。
2. 掌握 **方法覆蓋 (Method Overriding)** 與型態轉換。
3. 理解 **多型 (Polymorphism)** 的核心觀念與動態綁定。
4. 學習 **抽象類別 (Abstract Class)** 與 **介面 (Interface)** 的區別與應用。
5. 了解現代 Java 特性：密封類別 (Sealed Classes) 與介面進階功能。

2.1 虛虛實實：抽象與繼承

- **繼承 (Inheritance)**：子類別繼承父類別，重用代碼並延伸功能。
- **抽象 (Abstraction)**：虛化實作細節，僅定義必要的行為規格。

2.1.1 類別繼承 (Inheritance)

- 當類別 B 繼承類別 A (`B extends A`)：
  - B 具備 A 的特性（程式碼重用）。
  - B 可以擴充自己的特性。
- **術語**：
  - A：父類別 (Super Class / Parent Class)
  - B：子類別 (Sub Class / Child Class)

繼承中的建構子

- 子類別物件生成時，會自動呼叫父類別建構子。
- 如果父類別沒有預設建構子（不帶參數），子類別必須顯式使用 `super(...)` 。

```
public Bike(int speed, int seatHeight) {
    super(speed); // 呼叫父類別建構子
    this.seatHeight = seatHeight;
}
```

2.1.2 方法覆蓋 (Method Overriding)

- 子類別重新定義父類別中已存在的方法。
- 目的：提供更具體的實作。

```
class Animal {
    void speak() { System.out.println("Animal speaks"); }
}
class Dog extends Animal {
    @Override
    void speak() { System.out.println("Woof!"); }
}
```

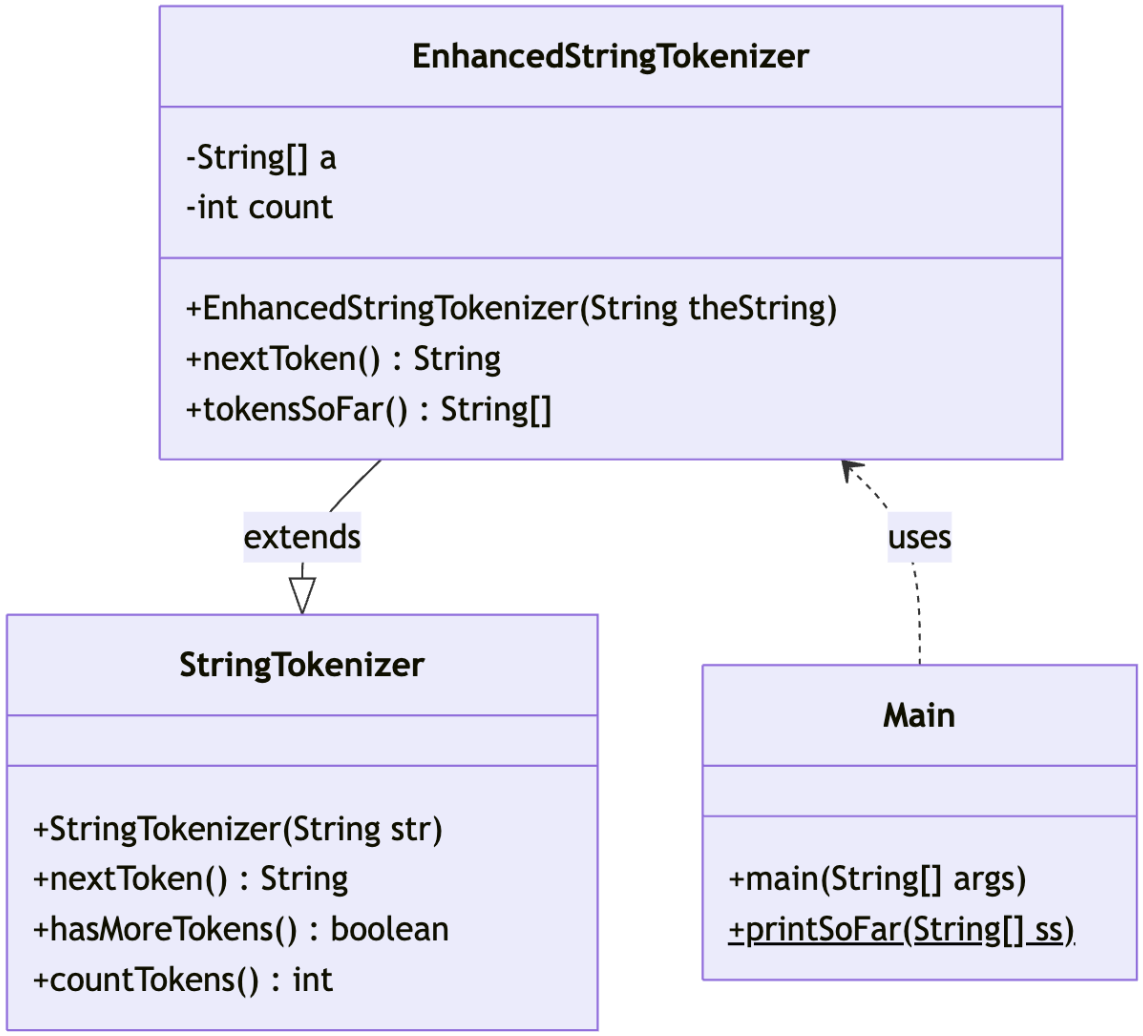
2.1.3 型態轉換 (Casting)

- **向上轉型 (Upcasting)**：子類給父類，自動轉換，安全。
  - `Person a = new Engineer();`
- **向下轉型 (Downcasting)**：父類給子類，需強制轉換，有風險。
  - `Engineer b = (Engineer) a;`
- **注意**：不正確的轉型會導致 `ClassCastException`。

2.1.6 現代 Java：密封類別 (Sealed Classes)

- 限制哪些類別可以繼承自己（Java 17+）。

```
public sealed class Shape permits Circle, Square { }
```


EnhancedStringTokenizer
-String[] a
-int count
+EnhancedStringTokenizer(String theString)
+nextToken() : String
+tokensSoFar() : String[]
extends
uses
StringTokenizer
+StringTokenizer(String str)
+nextToken() : String
+hasMoreTokens() : boolean
+countTokens() : int
Main
+main(String[] args)
+printSoFar(String[] ss)

❓ 觀念測驗 2.1

關於 Java 的類別繼承，以下哪一項是正確的？

A) Java 支援類別的多重繼承
B) `final` 關鍵字可用來防止類別被繼承
C) `private` 方法可以被子類別覆寫
D) 抽象類別無法包含已實作的方法

💡 解答 2.1

答案：B

- Java 不支援類別多重繼承（避免菱形問題）。
- `final` 類別不能有子類別。
- `private` 方法對子類別不可見，無法覆寫。

2.2 一法多形：多型 (Polymorphism)

- **多型**：允許將不同子類別物件視為父類別，實現「同一個介面，多種表現」。
- **動態綁定**：執行時期根據「物件實際型態」決定執行邏輯。

2.2.1 多型的核心觀念

1. **向上轉型**：前提。
2. **方法覆蓋**：不同實作。
3. **動態綁定 (Dynamic Binding)**：JVM 在執行時期決定呼叫哪個版本的方法。

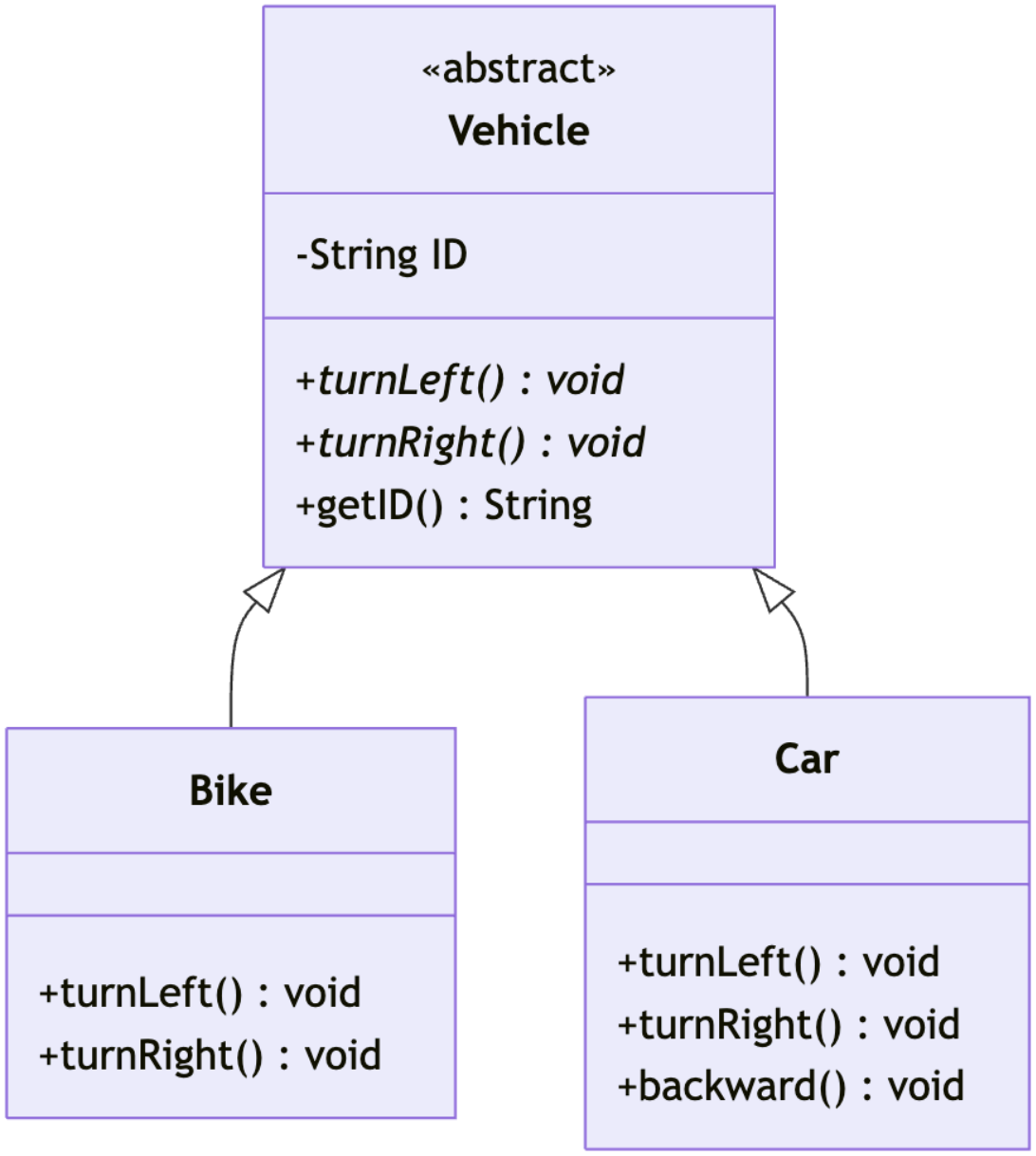
> **關鍵**：不要看變數宣告的型態，要看物件實際生成的型態。

2.2.3 多型容器 (Polymorphic Collections)

- 統一管理不同型態的子類別物件。

```
ArrayList<Shape> shapes = new ArrayList<>();
shapes.add(new Circle());
shapes.add(new Square());

for (Shape s : shapes) {
    s.draw(); // 動態綁定，畫出正確形狀
}
```


«abstract»
Vehicle
-String ID
+turnLeft() : void
+turnRight() : void
+getID() : String
Bike
+turnLeft() : void
+turnRight() : void
Car
+turnLeft() : void
+turnRight() : void
+backward() : void

❓ 觀念測驗 2.2

當呼叫 `Vehicle v = new Car(); v.turnLeft();` 時，執行的是哪個類別的方法？

A) `Vehicle` 的方法
B) `Car` 的方法
C) 會報錯，因為型態不符
D) 由編譯器在編譯時決定

💡 解答 2.2

答案：B

- 這是 **動態綁定**。
- 雖然變數 `v` 的編譯時期型態是 `Vehicle`，但執行時期實際上是 `Car` 物件。

2.3 無色無相：介面 (Interface)

- **介面**：定義物件「該做什麼」的純粹規格，不涉及實作細節。
- **解耦**：將功能需求與具體程式碼分離，提升系統彈性。

介面是什麼？

- **類別**：能做什麼 (Capability)。
- **介面**：該做什麼 (Specification)。
- 介面定義規格，不描述實作。
- 方法預設為 `public abstract` 。

2.3.3 多重繼承與介面

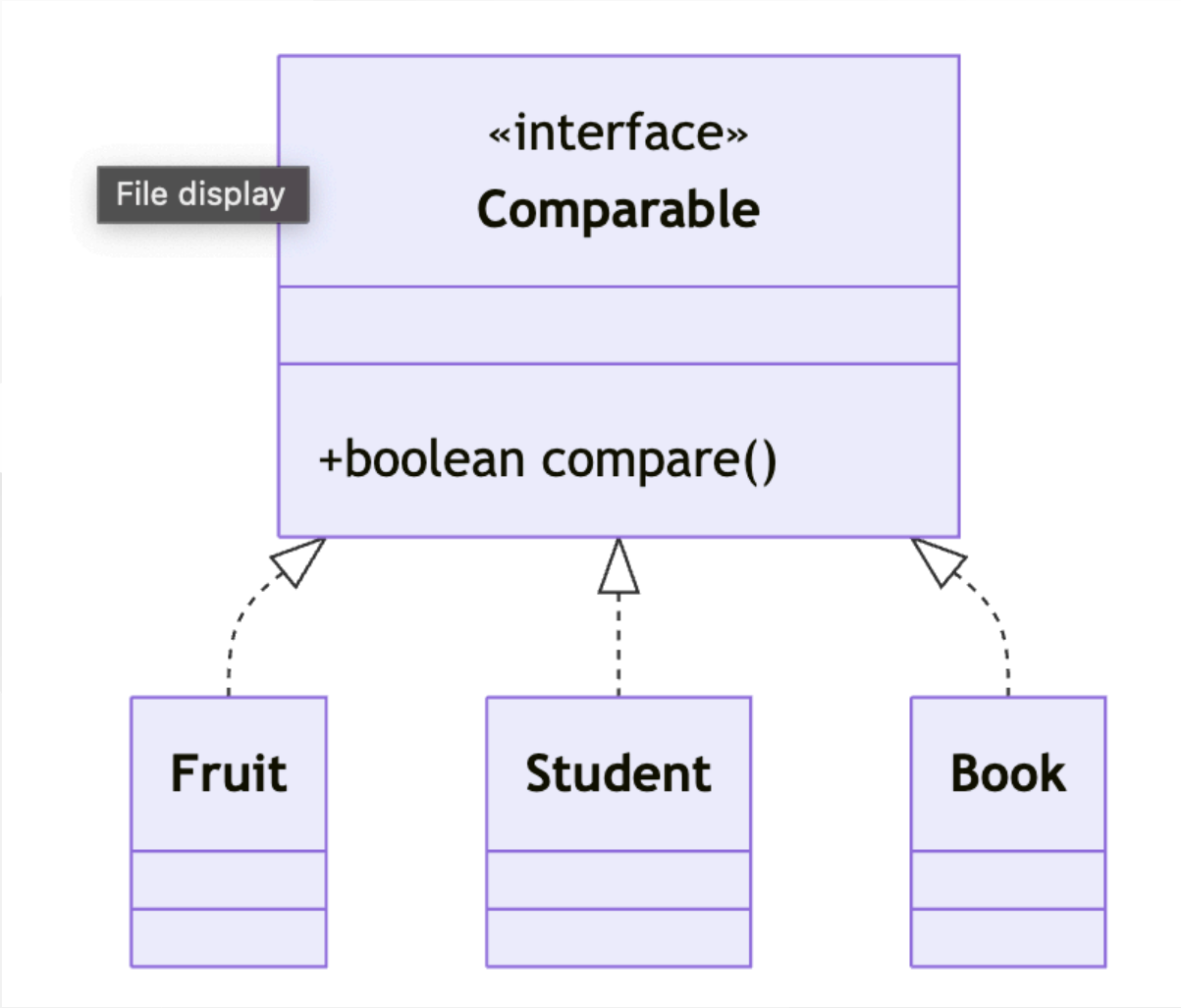
- Java 不允許多重繼承類別，但允許 **多重實作介面**。
- 解決了菱形繼承的實作衝突問題（因為介面方法通常沒有實作）。

2.3.4 介面的應用：Comparable

- 透過介面將「比較」行為一般化。

```
interface Comparable {
    public boolean betterThan(Comparable x);
}
```

- `Fruit` 或 `Student` 只要實作 `betterThan`，就能使用統一的 `Util.best()` 工具。


File display
«interface»
Comparable
+boolean compare()
Fruit
Student
Book

❓ 觀念測驗 2.3

關於介面 (Interface) 與 抽象類別 (Abstract Class) 的比較，何者正確？

A) 兩者都可以直接用 `new` 生成物件
B) 類別可以繼承多個抽象類別
C) 介面可以定義實體變數 (Instance Variables)
D) 類別可以實作多個介面

💡 解答 2.3

答案：D

- 抽象類別與介面都不能直接實例化。
- Java 僅支援單一繼承 (Extends)，但支援多重實作 (Implements)。
- 介面只能宣告常數，不能有實體變數。

課程總結

- **繼承**：`is-a` 關係，程式碼重用。
- **多型**：動態綁定，提升系統靈活性與擴充性 (開閉原則)。
- **介面**：定義行為規範，解耦實作。
- **抽象**：半成品，強制子類別完成特定行為。

Q & A

感謝大家的參與！